Give me 90 minutes, I will make Transformer click forever

Chapter 1: The Big Picture & Our Blueprint

You've likely used ChatGPT or a similar large language model. You type a question, and a coherent, well-written answer appears. It feels like magic. It can write poetry, debug code, and explain complex topics. But what if I told you that the core engine behind this apparent intelligence is not some unknowable black box?

What if I told you... this is the entire secret?

```python
# gpt2_min.py
import math
from dataclasses import dataclass
import torch
import torch.nn as nn
import torch.nn.functional as F

@dataclass
class GPTConfig:
    vocab_size: int
    block_size: int
    n_layer: int = 12
    n_head: int = 12
    n_embd: int = 768
    dropout: float = 0.1

class CausalSelfAttention(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        self.n_head, self.n_embd = config.n_head, config.n_embd
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd)
        self.resid_drop = nn.Dropout(config.dropout)
        self.register_buffer("bias", torch.tril(torch.ones(config.block_size, config.block_size)).view(1,
    def forward(self, x):
        B, T, C = x.size()
        qkv = self.c_attn(x)
        q, k, v = qkv.split(self.n_embd, dim=2)
        head_dim = C // self.n_head
        q = q.view(B, T, self.n_head, head_dim).transpose(1, 2)
        k = k.view(B, T, self.n_head, head_dim).transpose(1, 2)
        v = v.view(B, T, self.n_head, head_dim).transpose(1, 2)
        att = (q @ k.transpose(-2, -1)) * (1.0 / math.sqrt(head_dim))
        att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float("-inf"))
        att = F.softmax(att, dim=-1)
        y = att @ v
        y = y.transpose(1, 2).contiguous().view(B, T, C)
        return self.resid_drop(self.c_proj(y))

class MLP(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.fc = nn.Linear(config.n_embd, 4 * config.n_embd)
        self.proj = nn.Linear(4 * config.n_embd, config.n_embd)
        self.drop = nn.Dropout(config.dropout)
    def forward(self, x):
        return self.drop(self.proj(F.gelu(self.fc(x))))

class Block(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
```

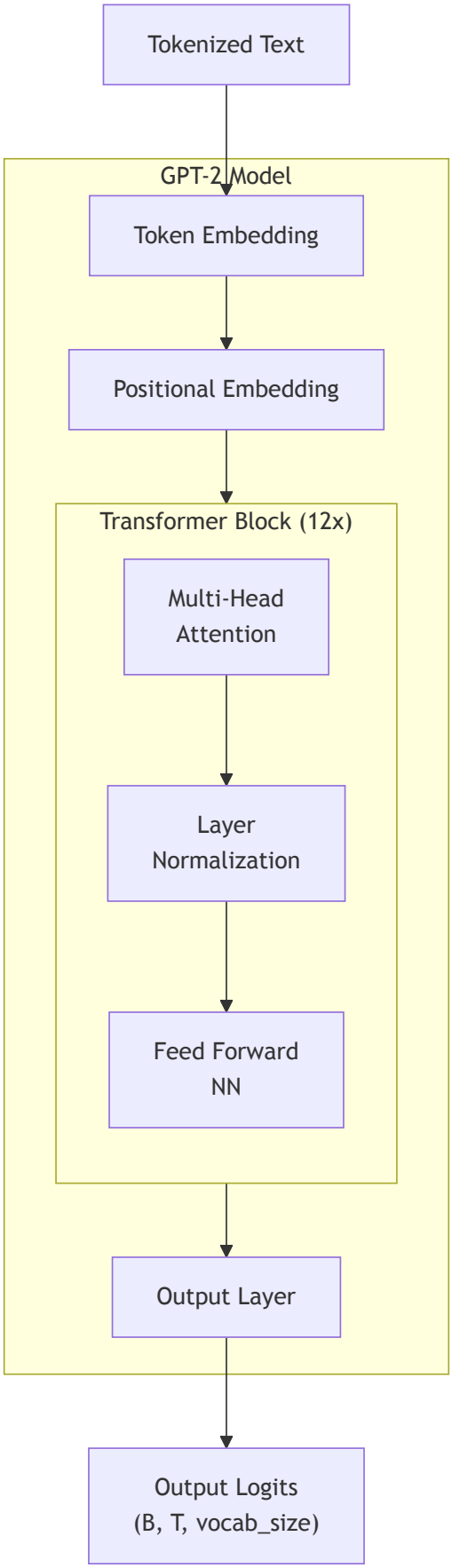
```
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)
    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x

class GPT2(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.config = config
        self.wte = nn.Embedding(config.vocab_size, config.n_embd)
        self.wpe = nn.Embedding(config.block_size, config.n_embd)
        self.drop = nn.Dropout(config.dropout)
        self.h = nn.ModuleList([Block(config) for _ in range(config.n_layer)])
        self.ln_f = nn.LayerNorm(config.n_embd)
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.lm_head.weight = self.wte.weight
    def forward(self, idx, targets=None):
        B, T = idx.size()
        pos = torch.arange(0, T, dtype=torch.long, device=idx.device).unsqueeze(0)
        x = self.wte(idx) + self.wpe(pos)
        x = self.drop(x)
        for block in self.h:
            x = block(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)
        loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1)) if targets is not None
        return logits, loss
```

That's it. These ~80 lines of Python and PyTorch contain all the fundamental architectural principles of a multi-billion dollar model like ChatGPT. There is no hidden magic.

In this tutorial, we are going to go through this file, line by line. You will understand not only **what** each line does, but **why** it's there—the core intuition behind it. By the end, the magic will dissolve into understandable, elegant engineering.

Our Promise and Roadmap

Our promise is simple: in the next 90 minutes, the Transformer will click for you forever. Our journey will follow this exact architecture:



This diagram is our roadmap.

1. **Input:** We start at the bottom with our "Tokenized Text".

2. **Embeddings:** We will first build the "Token Embedding" and "Positional Embedding" layers, which turn words and their positions into vectors.
3. **The Core Engine:** We will then build the "Transformer Block". This is the heart of the model, containing "Multi-Head Attention" and a "Feed Forward NN". As the "12 X" indicates, the model's power comes from stacking these blocks repeatedly.
4. **Output:** Finally, we will build the "Output Layer" that converts the final processed vectors back into a prediction.

The most important component we will build is the **Causal Self-Attention** mechanism, which follows this formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V$$

Don't worry if this looks intimidating. We will build up to it slowly until it becomes second nature.

Our Blueprint: The `GPTConfig`

Every construction project starts with a blueprint. For our model, that blueprint is the `GPTConfig` class. It's a simple container that holds all the key architectural parameters.

```
@dataclass
class GPTConfig:
    vocab_size: int
    block_size: int         # max sequence length (context window)
    n_layer: int = 12       # Number of Transformer Blocks to stack
    n_head: int = 12        # Number of attention "heads"
    n_embd: int = 768       # The dimensionality of our vectors
    dropout: float = 0.1
```

These are the knobs you can turn to change the size and power of the model.

Parameter	What it Controls	Intuition	Real GPT-2 (Small) Value
`vocab_size`	Vocabulary Size	How many unique words/tokens the model knows.	50257
`block_size`	Context Window	The maximum number of tokens the model can look at simultaneously.	1024
`n_layer`	Model **Depth**	The number of `Block`s stacked on top of each other. More layers = more powerful.	12
`n_head`	Model **Width**	The number of parallel "conversations" attention can have. More heads = more perspectives.	12
`n_embd`	Embedding Dimension	The "size" of the vectors representing each token.	768

By simply changing these numbers, you can create a tiny, toy model or the full-scale GPT-2. The underlying code for the architecture remains exactly the same.

With our blueprint in hand, we're ready to lay the first brick. In the next chapter, we will build the embedding layers that turn simple numbers into the rich vectors our model can understand.

Chapter 2: The Word-Vector Dictionary: Token Embeddings

Our journey begins with the first functional layer. We must convert the raw input—a sequence of token IDs—into something a neural network can process.

Let's look at our blueprint and pinpoint the component we're building.

```
class GPT2(nn.Module):
    def __init__(self, config):
        # We are building THIS line now.
        self.wte = nn.Embedding(config.vocab_size, config.n_embd) # Word Token Embedding
        self.wpe = nn.Embedding(...) # (Next chapter)

        # The rest of the model
        self.h = nn.ModuleList(...)
        self.ln_f = nn.LayerNorm(...)
        self.lm_head = nn.Linear(...)
```

The Input Problem: Meaningless Numbers

The input to our model is a tensor of token IDs, like `torch.tensor([[5, 21]])`. These are categorical numbers. The ID `21` doesn't have 4.2 times the "value" of ID `5`. The numerical distance between them is arbitrary and meaningless. A neural network, which relies on matrix multiplication and gradient descent, cannot learn from these raw IDs. They are just pointers.

The Goal: Mapping Words to a "Semantic Space"

Our goal is to create a learnable representation for each token. We want to map each token ID to a vector—a point in a high-dimensional space. The key idea is that the *location* of these points should be meaningful.

Analogy: A Color Space. Imagine we want to represent colors.

- **Bad way (Categorical ID):** `{"red": 1, "orange": 2, "blue": 8}`. The number `8` for "blue" has no relation to `1` for "red".
- **Good way (Vector/Coordinate):** Represent colors in a 2D space where the `x-axis` is "redness" and the `y-axis` is "blueness".
  - `red` might be `(0.9, 0.1)`
  - `orange` might be `(0.8, 0.2)` (close to red!)
  - `blue` might be `(0.1, 0.9)` (far from red!)

Now, the distance between points is meaningful! We are going to do the same thing, but for words, in a space with `n_embd` (e.g., 768) dimensions.

The Mechanism: A Learnable Coordinate Book

The `nn.Embedding` layer is this coordinate book. It is a simple lookup table stored as a single weight matrix. Let's build it and inspect its contents.

```
import torch
import torch.nn as nn

# A tiny config for our example
vocab_size = 10
n_embd = 3 # The number of dimensions in our "semantic space"

# The layer is our coordinate book
token_embedding_table = nn.Embedding(vocab_size, n_embd)

# The book itself is the `.weight` attribute. Each row is a word's coordinate.
print("Shape of our coordinate book:", token_embedding_table.weight.shape)
print("Content of the book (initially random coordinates):")
print(token_embedding_table.weight)
```

Output:

```
Shape of our coordinate book: torch.Size([10, 3])
Content of the book (initially random coordinates):
Parameter containing:
tensor([[-0.2185, -0.2291, -0.5435],  # Coordinate for token 0
        [ 0.6508,  0.4734, -0.4439],  # Coordinate for token 1
        [-0.3129,  1.6141,  0.2889],  # Coordinate for token 2
        ...
        [-1.0223, -0.2543, -0.3288]], # Coordinate for token 9
       requires_grad=True)
```

This is the core. We have a `(vocab_size, n_embd)` matrix. The `requires_grad=True` part is crucial: it means that during training, the model will learn the best possible coordinates for each word to minimize its prediction error.

The Power and The Insufficiency

Let's see the lookup in action and then discuss what this new representation gives us, and what it lacks.

```
B, T = 2, 4 # Batch, Time
idx = torch.randint(0, vocab_size, (B, T)) # A batch of token ID sequences

# --- The Lookup ---
# For each ID in `idx`, we retrieve its coordinate vector from the table
tok_emb = token_embedding_table(idx)

print("Input IDs shape:", idx.shape)
print("Output Vectors (Coordinates) shape:", tok_emb.shape)
```

Output:

```
Input IDs shape: torch.Size([2, 4])
Output Vectors (Coordinates) shape: torch.Size([2, 4, 3])
```

We transformed our `(B, T)` integer tensor into a `(B, T, n_embd)` float tensor.

So what have we gained?

We now have a **context-free representation** of each word. During training, the model learns to place words with similar meanings near each other. This is what enables the famous analogy `vector('King') - vector('Man') + vector('Woman') ≈ vector('Queen')`. The vector for 'King' captures a concept of "maleness" and "royalty" that can be manipulated mathematically.

But what is the insufficiency?

This representation has one massive flaw: it is **static and context-free**. The vector for the word "bank" is identical in these two sentences:

1. "I sat on the river **bank**."
2. "I withdrew money from the **bank**."

The lookup table gives us a powerful starting point, but it's just a dictionary. It doesn't know anything about the words surrounding it.

This is the central problem the Transformer architecture is designed to solve. **How can a token's vector representation be dynamically adjusted based on its context?**

To even begin to answer that, we first need to give the model a sense of order. That is the subject of the next chapter: Positional Embeddings.

Chapter 3: Giving the Model a Sense of Order: Positional Embeddings

In the last chapter, we created a "dictionary" that maps each word to a context-free vector. This vector represents the word's general meaning. However, our model still sees the input as just a collection—a "bag"—of these vectors. It has no idea about their order.

Language is all about order. "Dog bites man" is not the same as "Man bites dog." We must provide this crucial ordering information to the model.

Let's locate our next component in the blueprint.

```
class GPT2(nn.Module):
    def __init__(self, config):
        self.wte = nn.Embedding(...) # (Done)

        # We are building THIS line now.
        self.wpe = nn.Embedding(config.block_size, config.n_embd) # Positional Embedding

        self.h = nn.ModuleList(...)
        self.ln_f = nn.LayerNorm(...)
        self.lm_head = nn.Linear(...)
```

The Problem: A Bag of Words

Our current output is a tensor of shape `(B, T, C)`, where `C` is `n_embd`. For a sequence like `["Man", "bites", "dog"]`, the model receives a set of three vectors: `{vector("Man"), vector("bites"), vector("dog")}`. If we shuffled the input, the model would receive the exact same set of vectors, just in a different order along the `T` dimension. The core processing layers (the Transformer blocks) are designed to be order-invariant, so without modification, they would produce the same result.

We need to explicitly "stamp" each token's vector with its position.

The Solution: A Learnable "Position Vector"

The solution used in GPT is wonderfully simple. Just as we learned a unique vector for each *word*, we will also learn a unique vector for each *position*.

- We'll have a vector that means "I am at the 1st position."
- We'll have another vector that means "I am at the 2nd position."
- ...and so on, up to the maximum sequence length (`block_size`).

This is another `nn.Embedding` layer, but this time it's a lookup table for positions, not words. Its "vocabulary size" is the `block_size`. We then add this position vector to the corresponding token vector.

Why does adding them work?
Because the token and positional embeddings exist in the same high-dimensional space, the model can learn to interpret the combined vector. During training, it learns to create positional vectors such that adding `vector(pos=N)` to `vector(word=W)` produces a unique representation that distinguishes it from the same word at a different position. The network learns to "understand" this composition.

The Code: Building and Combining

Let's implement this. The key is to generate a sequence of position indices `(0, 1, 2, ...)` and use those to look up the positional vectors.

```python
import torch
import torch.nn as nn

# --- Our Config ---
B, T, C = 2, 5, 3  # Batch, Time (sequence length), Channels (n_embd)
vocab_size = 10
block_size = 8     # Our model's max sequence length is 8

# --- The Layers ---
token_embedding_table = nn.Embedding(vocab_size, C)
position_embedding_table = nn.Embedding(block_size, C)

# --- The Input Data ---
idx = torch.randint(0, vocab_size, (B, T)) # Shape (2, 5)

# --- Step 1: Get Token Embeddings (as before) ---
tok_emb = token_embedding_table(idx) # Shape (B, T, C) -> (2, 5, 3)

# --- Step 2: Get Positional Embeddings ---
# Note: Our input sequence length T=5, but block_size=8. This is fine.
# We just need the positions for our current sequence.
pos = torch.arange(0, T, dtype=torch.long) # Shape (T) -> tensor([0, 1, 2, 3, 4])
pos_emb = position_embedding_table(pos) # Shape (T, C) -> (5, 3)

# --- Step 3: Combine them via addition ---
# (B, T, C) + (T, C) --(broadcasts)--> (B, T, C)
x = tok_emb + pos_emb
print("Shape of token embeddings:", tok_emb.shape)
print("Shape of positional embeddings:", pos_emb.shape)
print("Shape of final combined embeddings:", x.shape)
```

Output:

```
Shape of token embeddings: torch.Size([2, 5, 3])
Shape of positional embeddings: torch.Size([5, 3])
Shape of final combined embeddings: torch.Size([2, 5, 3])
```

Answering a key question: What if the input sequence is shorter than `block_size` ?

This is the normal case! As you see above, our `block_size` is `8` , but our input sequence length `T` is only `5` . The code `torch.arange(0, T, ...)` handles this perfectly. We only generate and look up the positional embeddings for the sequence length we are currently processing. We never use the full `block_size` unless our input is that long.

We have now prepared the input for the main event. Our tensor `x` contains vectors that are aware of both the token's identity and its absolute position in the sequence.

This is the input that will flow into the stack of Transformer blocks. It's time to build the heart of the machine: Self-Attention.

Chapter 4: The Heart of the Transformer: Self-Attention Intuition

In the last chapter, we established that our model gives the same starting vector to a word regardless of its context. This is a problem for ambiguous words. To make this crystal clear, we'll use simplified sentences.

The Problem: Consider the word "**crane**".

1. "**Crane** ate fish." (crane = a bird)
2. "**Crane** lifted steel." (crane = a machine)

The initial vector for "crane" is identical in both sentences. Self-attention must update this vector based on the neighbors (`ate` vs. `lifted`) to resolve the ambiguity.

Let's see how this works using the attention formula as our map:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

The Q, K, and V Vector Spaces

Before we jump in, let's clarify what Q, K, and V represent.

- **Query (Q) and Key (K) Space:** Think of this as the "matching" or "searching" space. Q is the probe, K is the label. They **must** have the same number of dimensions so we can compute their dot product to get a similarity score.
- **Value (V) Space:** Think of this as the "information" or "payload" space. This is the actual substance that gets passed along once a match is found. In our GPT-2 architecture, we set `d_v = d_k = C/n_head` for simplicity. Note that architecturally, `d_v` doesn't have to equal `d_k` - what matters is that after concatenating all heads, the total dimension equals `C` (i.e., `n_head * d_v = C`), ensuring the output has the same dimensions as the input for residual connections.

This raises a crucial question: **We already have an input vector `x` for each token. Why do we need to create a separate Value (`v`) vector? Isn't `x` already the 'information'?**

The answer is that the raw information is not always the best information to share. The `v` vector is a *transformed* version of `x` , specifically packaged for other tokens to consume.

Let's use an analogy. Imagine a token is a professional at a conference.

Aspect	Input Vector (`x`)	Value Vector (`v`)
Role	Raw, Complete Information	Packaged, Consumable Information
Analogy	Your entire knowledge & resume	Your prepared "elevator pitch"
Purpose	The starting point for computation	The payload to be aggregated by other tokens
How it's Made	Output of embedding layers	A **learned transformation** of `x` (`v = x @ W_v`)

The model *learns* the best "elevator pitch" (`v`) for each word. This gives it the flexibility to emphasize or de-emphasize parts of its raw knowledge (`x`) to be most helpful to its neighbors.

Now, let's proceed with our example. We'll use a 2D space where the dimensions are dead simple:

- **Dimension 1:** Represents "Is it an **Animal**?"
- **Dimension 2:** Represents "Is it a **Machine**?"

The ambiguous word "crane" will have vectors balanced between these possibilities.

Token	Q - "I'm looking for..."	K - "I am..."	V - "I offer this info..."
ate	...	`[0.9, 0.1]` (High Animal)	`[0.9, 0.1]`
fish	...	`[0.9, 0.1]` (High Animal)	`[0.8, 0.2]`
lifted	...	`[0.1, 0.9]` (High Machine)	`[0.1, 0.9]`
steel	...	`[0.1, 0.9]` (High Machine)	`[0.2, 0.8]`
crane	**`[0.7, 0.7]`**	**`[0.7, 0.7]`**	**`[0.5, 0.5]`** (Ambiguous)

Sentence 1: "Crane ate fish"

1. Scoring (`QK^T`): The `crane` token uses its query `[0.7, 0.7]` to probe all keys in the sentence.

- **Score(`crane -> crane`):** `[0.7, 0.7]` · `[0.7, 0.7]` = 0.49 + 0.49 = **0.98**
- **Score(`crane -> ate`):** `[0.7, 0.7]` · `[0.9, 0.1]` = 0.63 + 0.07 = **0.70**
- **Score(`crane -> fish`):** `[0.7, 0.7]` · `[0.9, 0.1]` = 0.63 + 0.07 = **0.70**

2. Normalizing (`softmax`): The raw scores `[0.98, 0.70, 0.70]` are converted to percentages.

- Attention Weights: `[0.4, 0.3, 0.3]`
  This means `crane` will construct its new self by listening 40% to its original self, 30% to `ate` , and 30% to `fish` .

3. Aggregating (`...V`): The new vector for `crane` is a weighted sum of the **Values**.

- `New_Vector(crane) = 0.4*V(crane) + 0.3*V(ate) + 0.3*V(fish)`
- `New_Vector(crane) = 0.4*[0.5, 0.5] + 0.3*[0.9, 0.1] + 0.3*[0.8, 0.2]`
- `New_Vector(crane) = [0.20, 0.20] + [0.27, 0.03] + [0.24, 0.06] =` **`[0.71, 0.29]`**

The result is a new "crane" vector that is heavily skewed towards **Dimension 1 (Animal)**. The context from `ate` and `fish` has resolved the ambiguity. It's a bird.

Sentence 2: "Crane lifted steel"

1. Scoring (`QK^T`): `crane` uses the *exact same query* `[0.7, 0.7]` on its new neighbors.

- **Score(`crane -> crane`):** `[0.7, 0.7]` · `[0.7, 0.7]` = **0.98**
- **Score(`crane -> lifted`):** `[0.7, 0.7]` · `[0.1, 0.9]` = 0.07 + 0.63 = **0.70**
- **Score(`crane -> steel`):** `[0.7, 0.7]` · `[0.1, 0.9]` = 0.07 + 0.63 = **0.70**

2. Normalizing (`softmax`): The raw scores `[0.98, 0.70, 0.70]` are identical to before.

- Attention Weights: `[0.4, 0.3, 0.3]`
  The percentages are the same, but they now apply to a different set of tokens!

3. Aggregating (`...V`):

- `New_Vector(crane) = 0.4*V(crane) + 0.3*V(lifted) + 0.3*V(steel)`
- `New_Vector(crane) = 0.4*[0.5, 0.5] + 0.3*[0.1, 0.9] + 0.3*[0.2, 0.8]`
- `New_Vector(crane) = [0.20, 0.20] + [0.03, 0.27] + [0.06, 0.24] = [0.29, 0.71]`

The result is a vector now heavily skewed towards **Dimension 2 (Machine)**. The exact same initial "crane" vector has been transformed into a completely different, context-aware vector because it listened to different dominant neighbors.

This is the power of self-attention. Now that the intuition is solid, we can finally implement it with matrices.

Chapter 5: Implementing Scaled Dot-Product Attention

We've built the intuition for self-attention. Now, we will translate that exact process into matrix operations using PyTorch. By the end of this chapter, you will have implemented the core attention formula and encapsulated it into a reusable `nn.Module` .

Our map for this chapter is the formula itself:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

First, we will build this with raw tensors to see every number. Then, we will put that logic inside our official `CausalSelfAttention` class.

Part 1: The Raw Tensor Walkthrough

Let's use a simple sentence: "**A crane ate fish**". We now have 4 tokens (`T=4`) and our toy embedding dimension is 2 (`C=2`). We'll process one sentence at a time (`B=1`).

The Input (`x`): Raw, Context-Free Embeddings
This is the tensor from our embedding layers. Dim1="Object-like", Dim2="Action-like".

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math
from dataclasses import dataclass

B, T, C = 1, 4, 2  # Batch, Time (sequence length), Channels (embedding dim)
x = torch.tensor([
    [[0.1, 0.1],   # A
     [1.0, 0.2],   # crane (mostly object, slightly action)
     [0.1, 0.9],   # ate (mostly action)
     [0.8, 0.0]]   # fish (purely object)
]).float()
```

Step 1: Projecting `x` into Q, K, and V

To get our Query, Key, and Value vectors, we use **learnable** linear transformations. These `nn.Linear` layers are the "brains" of the operation; their weights are updated during training. For this tutorial, we will set them manually to see the logic clearly.

```
# The learnable components
q_proj = nn.Linear(C, C, bias=False)
k_proj = nn.Linear(C, C, bias=False)
v_proj = nn.Linear(C, C, bias=False)

# Manually set weights for this tutorial
torch.manual_seed(42)
q_proj.weight.data = torch.randn(C, C)
k_proj.weight.data = torch.randn(C, C)
v_proj.weight.data = torch.randn(C, C)

# --- Perform the projections ---
q = q_proj(x)
k = k_proj(x)
v = v_proj(x)
```

Let's track our tensor shapes and their meaning.

Variable	Shape `(B, T, C)`	Meaning
`x`	`(1, 4, 2)`	The batch of raw input vectors.
`q`	`(1, 4, 2)`	The "Query" vector for each of the 4 tokens.
`k`	`(1, 4, 2)`	The "Key" vector for each of the 4 tokens.
`v`	`(1, 4, 2)`	The "Value" vector for each of the 4 tokens.

Step 2: Calculate Attention Scores (`q @ k.transpose`)

This is the core of the communication. We need to compute the dot product of every token's query with every other token's key. We can do this with a single, efficient matrix multiplication.

- `q` has shape `(1, 4, 2)` .
- `k` has shape `(1, 4, 2)` .
- To multiply them, we need to make their inner dimensions match. We use `.transpose(-2, -1)` to swap the last two dimensions of `k` .
- `k.transpose(-2, -1)` results in a shape of `(1, 2, 4)` .
- The multiplication is `(1, 4, 2) @ (1, 2, 4)` , which results in a `(1, 4, 4)` matrix.

```
# --- Score Calculation ---
scores = q @ k.transpose(-2, -1)

print("--- Raw Scores (Attention Matrix) ---")
print(scores.shape)
print(scores)
```

Output:

```
--- Raw Scores (Attention Matrix) ---
torch.Size([1, 4, 4])
tensor([[[ 0.0531,  0.4137,  0.1802,  0.2721],   # "A" scores for (A, crane, ate, fish)
         [ 0.1782,  1.3888,  0.6053,  0.9101],   # "crane" scores for (A, crane, ate, fish)
         [ 0.0618,  0.4815,  0.2098,  0.3151],   # "ate" scores for (A, crane, ate, fish)
         [ 0.1260,  0.9822,  0.4280,  0.6433]]])  # "fish" scores for (A, crane, ate, fish)
```

This `(4, 4)` matrix holds the raw compatibility scores. For example, the query for "crane" (row 1) has the highest compatibility with the key for "crane" (column 1), which is `1.3888`.

Step 3 & 4: Scale and Softmax

We scale the scores for stability, then use `softmax` to turn them into attention weights that sum to 1 for each row.

```
d_k = k.size(-1)
scaled_scores = scores / math.sqrt(d_k)
attention_weights = F.softmax(scaled_scores, dim=-1) # Softmax along the rows
```

Step 5: Aggregate the Values (`attention_weights @ v`)

Now we use our weights to create a weighted average of the `Value` vectors.

- `attention_weights` has shape `(1, 4, 4)`.
- `v` has shape `(1, 4, 2)`.
- The multiplication `(1, 4, 4) @ (1, 4, 2)` produces a final tensor of shape `(1, 4, 2)`.

```
# --- Value Aggregation ---
output = attention_weights @ v

print("\n--- Final Output (Context-Aware Vectors) ---")
print(output.shape)
print(output)
```

Output:

```
--- Final Output (Context-Aware Vectors) ---
torch.Size([1, 4, 2])
tensor([[[ 0.0652, -0.1691],
         [ 0.1147, -0.2974],
         [ 0.0768, -0.1991],
         [ 0.1005, -0.2607]]])
```

Here is a summary of the tensor transformations:

Step	Operation	Input Shapes	Output Shape `(B, T, ...)`	Meaning
1	`q_proj(x)` etc.	`(1, 4, 2)`	`(1, 4, 2)`	Create Q, K, V for each token
2	`q @ k.T`	`(1, 4, 2)` & `(1, 2, 4)`	`(1, 4, 4)`	Raw compatibility scores
3	`/ sqrt(d_k)`	`(1, 4, 4)`	`(1, 4, 4)`	Stabilized scores
4	`softmax`	`(1, 4, 4)`	`(1, 4, 4)`	Attention probabilities
5	`att @ v`	`(1, 4, 4)` & `(1, 4, 2)`	`(1, 4, 2)`	Context-aware output vectors

Success! We have taken our raw input `x` and produced a new tensor `output` of the exact same shape, where each token's vector has been updated with information from its neighbors.

Part 2: Encapsulating the Logic in an `nn.Module`

The raw tensor walkthrough is fantastic for understanding the mechanics. In practice, we package this logic into a reusable class. This makes our code clean, organized, and easy to integrate into a larger model.

Here is the complete, encapsulated code for a single attention head. By the end of this section, every single line will be crystal clear.

```python
# The final, reusable PyTorch module
class SingleHeadSelfAttention(nn.Module):
    def __init__(self, config):
        """
        Initializes the layers needed for self-attention.
        """
        super().__init__()
        # The single, fused linear layer for Q, K, V
        self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd, bias=False)

    def forward(self, x):
        """
        Defines the data flow through the module.
        Input x shape: (B, T, C)
        """
        B, T, C = x.size()

        # 1. Get Q, K, V from a single projection and split them
        qkv = self.c_attn(x)
        q, k, v = qkv.split(C, dim=2)

        # 2. Calculate attention weights
        # (B, T, C) @ (B, C, T) -> (B, T, T)
        scaled_scores = (q @ k.transpose(-2, -1)) / math.sqrt(k.size(-1))
        attention_weights = F.softmax(scaled_scores, dim=-1)

        # 3. Aggregate values
        # (B, T, T) @ (B, T, C) -> (B, T, C)
        output = attention_weights @ v

        return output
```

Now, let's break down how this elegant code achieves the exact same result as our manual, step-by-step process.

The `__init__` Method: The Fused Linear Layer

The `__init__` method sets up the building blocks. Here, we only need one.

```python
self.c_attn = nn.Linear(config.n_embd, 3 * config.n_embd, bias=False)
```

In our manual walkthrough, we used three separate `nn.Linear` layers. This single line is a common and highly efficient optimization that achieves the same goal.

Our Manual Walkthrough (Conceptually Clear)	Fused Layer (Computationally Efficient)
`q_proj = nn.Linear(C, C)`	
`k_proj = nn.Linear(C, C)`	`c_attn = nn.Linear(C, 3*C)`
`v_proj = nn.Linear(C, C)`	

Instead of three smaller matrix multiplications, the GPU can perform one larger, faster matrix multiplication. The `bias=False` argument is a common simplification used in minimal implementations like NanoGPT. Note that the original GPT-2 implementation does include biases in its linear projections.

The `forward` Method: From Fused to Final Output

The `forward` method is the heart of the module, defining the data's journey.

1. Projection and Splitting

```
qkv = self.c_attn(x)
q, k, v = qkv.split(C, dim=2)
```

- `self.c_attn(x)` : We pass our input `x` (shape `B, T, C`) through the fused layer, resulting in a `qkv` tensor of shape `(B, T, 3*C)` .
- `qkv.split(C, dim=2)` : This is the clever part. The `.split()` function carves up the tensor. We tell it: "Along dimension 2 (the last dimension), create chunks of size `C` ." Since the total dimension is `3*C` , this gives us exactly three tensors, each with the desired shape of `(B, T, C)` , which we assign to `q` , `k` , and `v` .

2. Calculating Attention Weights

```
scaled_scores = (q @ k.transpose(-2, -1)) / math.sqrt(k.size(-1))
attention_weights = F.softmax(scaled_scores, dim=-1)
```

This is a direct, one-to-one implementation of the mathematical formula.

- `k.transpose(-2, -1)` swaps the `T` and `C` dimensions of the Key tensor to prepare for matrix multiplication.
- `q @ ...` performs the dot product, resulting in the raw score matrix of shape `(B, T, T)` .
- `/ math.sqrt(k.size(-1))` performs the scaling for stability.
- `F.softmax(...)` converts the raw scores into a probability distribution along each row.

3. Aggregating Values

```
output = attention_weights @ v
```

Finally, we perform the last matrix multiplication. The attention weights `(B, T, T)` are multiplied with the Value vectors `(B, T, C)` , resulting in our final output tensor of shape `(B, T, C)` .

Proof of Equivalence

To prove this class is identical to our manual work, we can instantiate it and manually load the weights from our `q_proj` , `k_proj` , and `v_proj` layers into the single `c_attn` layer.

```
# Let's verify the logic
@dataclass
class GPTConfig: n_embd: int
model = SingleHeadSelfAttention(GPTConfig(n_embd=C))

# The c_attn layer's weight matrix is shape (3*C, C). Our separate weights
# are each (C, C). We concatenate them along dim=0 to get (3*C, C).
model.c_attn.weight.data = torch.cat(
    [q_proj.weight.data, k_proj.weight.data, v_proj.weight.data], dim=0
)

# Run the model
model_output = model(x)

# 'output' is the tensor from our manual walkthrough in Part 1
print("Are the outputs the same?", torch.allclose(output, model_output))
```

Output:

```
Are the outputs the same? True
```

It works perfectly. We have successfully implemented the core of self-attention and formalized it in a clean, reusable module.

However, our model has a flaw for language generation: tokens can see into the future. Our current attention matrix allows this. We will fix this next by adding a causal mask.

Chapter 6: Don't Look Ahead! Implementing the Causal Mask

We have built a powerful attention mechanism that allows tokens to communicate. However, it has a critical flaw for our purpose: it's a time-traveler.

The Problem: Cheating by Looking at the Future

GPT is an **autoregressive** model, meaning it generates text one token at a time. When it's trying to predict the next word in the sentence "A crane ate...", its decision should be based *only* on the tokens it has seen so far: "A", "crane", and "ate". It cannot be allowed to see the actual answer, "fish".

Let's look at the attention weight matrix we calculated in the last chapter:

```
# From Chapter 5
tensor([[[0.37, 0.32, 0.31, ...],  # "A" attends to all 4 tokens
         [0.31, 0.37, 0.32, ...],  # "crane" attends to all 4 tokens
         [0.36, 0.31, 0.33, ...],  # "ate" attends to all 4 tokens
         ...                      # "fish" attends to all 4 tokens
        ]]])
```

This is a problem. The token "A" (at position 0) is gathering information from "crane" (position 1), "ate" (position 2), and "fish" (position 3). When predicting the word that comes after "A", this is cheating.

A token at position `t` must only be allowed to communicate with tokens at positions `0, 1, ..., t`. It cannot see tokens at `t+1, t+2, ...`.

The Solution: The Causal Mask

The solution is elegant. We will modify the attention score matrix *before* applying the softmax function. We will "mask out" all the future positions by setting their scores to negative infinity (`-inf`).

Why `-inf`? Because the `softmax` function involves an exponential: `e^x / sum(e^x)`. The exponential of negative infinity, `e^-inf`, is effectively zero. This forces the attention weights for all future tokens to become `0`, preventing any information flow.

Part 1: The Raw Tensor Walkthrough

Let's pick up exactly where we left off, with our `scaled_scores` matrix from Chapter 5.

```
# This is the scaled_scores tensor from the end of the last chapter
# Shape (B, T, T) -> (1, 4, 4)
scaled_scores = torch.tensor([[
    [ 0.0375,  0.2925,  0.1274,  0.1924],
    [ 0.1260,  0.9822,  0.4280,  0.6433],
    [ 0.0437,  0.3405,  0.1484,  0.2228],
    [ 0.0891,  0.6945,  0.3023,  0.4549]
]])
```

Step 1: Create the Mask
We need a mask that allows a token to see itself and the past, but not the future. A lower-triangular matrix is perfect for this. We can create one easily with `torch.tril`.

```
# T=4 for our sentence "A crane ate fish"
T = 4
mask = torch.tril(torch.ones(T, T))
print("--- The Mask ---")
print(mask)
```

Output:

```
--- The Mask ---
tensor([[1., 0., 0., 0.],
        [1., 1., 0., 0.],
        [1., 1., 1., 0.],
        [1., 1., 1., 1.]])
```

Look at the rows.

- Row 0 ("A") can only see column 0 ("A").

- Row 1 ("crane") can see column 0 ("A") and 1 ("crane").
- And so on. The zeros in the upper-right triangle represent the "future" connections that we must block.

Step 2: Apply the Mask

We use the PyTorch function `masked_fill` to apply our mask. This function will replace all values in `scaled_scores` with `-inf` wherever the corresponding position in our `mask` is `0`.

```
masked_scores = scaled_scores.masked_fill(mask == 0, float('-inf'))
print("\n--- Scores After Masking ---")
print(masked_scores)
```

Output:

```
--- Scores After Masking ---
tensor([[[ 0.0375,    -inf,    -inf,    -inf],
         [ 0.1260,  0.9822,    -inf,    -inf],
         [ 0.0437,  0.3405,  0.1484,    -inf],
         [ 0.0891,  0.6945,  0.3023,  0.4549]]])
```

Perfect! All the scores corresponding to future positions have been replaced.

Step 3: Re-run Softmax

Now, let's apply softmax to these masked scores and see the result.

```
attention_weights = F.softmax(masked_scores, dim=-1)
print("\n--- Final Causal Attention Weights ---")
print(attention_weights.data.round(decimals=2))
```

Output:

```
--- Final Causal Attention Weights ---
tensor([[[1.0000, 0.0000, 0.0000, 0.0000],
         [0.2995, 0.7005, 0.0000, 0.0000],
         [0.3129, 0.3807, 0.3064, 0.0000],
         [0.2186, 0.3999, 0.2445, 0.1370]]])
```

This is the "Aha!" moment. The upper-right triangle of our attention matrix is now all zeros.

- "A" can only attend to itself (100%).
- "crane" attends to "A" (30%) and "crane" (70%).
- "ate" attends to "A", "crane", and "ate".
  Information can now only flow from the past to the present.

Attention Type	**"crane" attends to "fish"?**	**"ate" attends to "fish"?**
Unmasked (Ch 5)	Yes	Yes
Causal (Ch 6)	**No (0%)**	**No (0%)**

Part 2: Encapsulating in the `nn.Module`

Now, let's add this logic to our `CausalSelfAttention` class from the `gpt2_min.py` file.

The `__init__` Method: `register_buffer`

We need to store our mask as part of the module. We use `register_buffer` for this.

```
class CausalSelfAttention(nn.Module):
    def __init__(self, config):
        super().__init__()
        # ... (c_attn layer from before)

        # We register the mask as a "buffer"
        self.register_buffer(
            "bias", # name of the buffer
            torch.tril(torch.ones(config.block_size, config.block_size))
            .view(1, 1, config.block_size, config.block_size)
        )
```

Why `register_buffer` ? A buffer is a tensor that is part of the model's state (like weights), so it gets moved to the GPU with `.to(device)` . However, it is **not** a parameter that gets updated by the optimizer during training. This is perfect for our fixed causal mask.

The `.view(1, 1, ...)` part is to add extra dimensions for broadcasting, which will be essential when we add Multi-Head Attention in the next chapter.

The `forward` Method: Adding the Masking Step

The `forward` pass is updated with one crucial line before the softmax.

```
    def forward(self, x):
        B, T, C = x.size()
        # ... (get q, k, v as before)

        scaled_scores = (q @ k.transpose(-2, -1)) / math.sqrt(k.size(-1))

        # --- THE NEW LINE ---
        # We slice the stored mask to match the sequence length T of our input
        scaled_scores = scaled_scores.masked_fill(self.bias[:, :, :T, :T] == 0, float("-inf"))

        attention_weights = F.softmax(scaled_scores, dim=-1)
        output = attention_weights @ v
        return output
```

We have now built a fully functional, **causal**, single-headed attention mechanism. It can learn to find context, but it can no longer cheat by looking into the future.

The final piece of the puzzle is to make it even more powerful by allowing it to have multiple "conversations" at once. This is the goal of our next chapter: Multi-Head Attention.

Chapter 7: Many Conversations at Once: Multi-Head Attention

So far, we have built a single, causal self-attention mechanism. It's like having one person in a meeting who is responsible for figuring out all the relationships between words. This is a lot of pressure. A single attention mechanism has to learn to focus on syntactic relationships ("is this an adjective modifying a noun?"), semantic relationships ("are these words related in meaning?"), and other patterns, all at once.

The Idea: Parallel Conversations

What if, instead of one overworked attention mechanism, we could have several working in parallel?

This is the core idea of **Multi-Head Attention**. We will split our embedding dimension `C` into smaller chunks, called "heads". Each head will be its own independent attention mechanism, complete with its own Q, K, and V projections.

- **Head 1** might learn to focus on verb-object relationships.
- **Head 2** might learn to focus on which pronouns refer to which nouns.
- **Head 3** might learn to track long-range dependencies in the text.
- ...and so on.

Each head conducts its own "conversation" and produces its own context-aware output vector. At the end, we simply concatenate the results from all the heads and pass them through a final linear layer to combine the insights.

Part 1: The Raw Tensor Walkthrough

Let's start with our `q`, `k`, and `v` tensors from Chapter 5.

- **Shape:** `(B, T, C)` -> `(1, 4, 768)` (Let's use a more realistic `C` for this example).
- **`n_head`:** Let's say we want `12` attention heads.
- **`head_dim`:** The dimension of each head will be `C / n_head`, which is `768 / 12 = 64`.

Step 1: Splitting `C` into `n_head` and `head_dim`
Our current `q` tensor has shape `(1, 4, 768)`. We need to reshape it so that the 12 heads are explicit. The target shape is `(B, n_head, T, head_dim)` or `(1, 12, 4, 64)`.

This is done with a sequence of `view()` and `transpose()` operations.

```
# --- Configuration ---
B, T, C = 1, 4, 768
n_head = 12
head_dim = C // n_head # 768 // 12 = 64

# --- Dummy Q, K, V tensors with realistic shapes ---
q = torch.randn(B, T, C)
k = torch.randn(B, T, C)
v = torch.randn(B, T, C)

# --- Reshaping Q ---
# 1. Start with q: (B, T, C) -> (1, 4, 768)
# 2. Reshape to add the n_head dimension
q_reshaped = q.view(B, T, n_head, head_dim) # (1, 4, 12, 64)
# 3. Transpose to bring n_head to the front
q_final = q_reshaped.transpose(1, 2) # (1, 12, 4, 64)

print("Original Q shape:", q.shape)
print("Final reshaped Q shape:", q_final.shape)
```

Output:

```
Original Q shape: torch.Size([1, 4, 768])
Final reshaped Q shape: torch.Size([1, 12, 4, 64])
```

We do the exact same reshaping for `k` and `v`. Now, PyTorch's broadcasting capabilities will treat the `n_head` dimension as a new "batch" dimension. All our subsequent attention calculations will be performed independently for all 12 heads at once.

Step 2: Run Attention in Parallel

Our attention formula remains the same, but now it operates on tensors with an extra `n_head` dimension.

```
# Reshape k and v as well
k_final = k.view(B, T, n_head, head_dim).transpose(1, 2) # (1, 12, 4, 64)
v_final = v.view(B, T, n_head, head_dim).transpose(1, 2) # (1, 12, 4, 64)

# --- Attention Calculation ---
# (B, nh, T, hd) @ (B, nh, hd, T) -> (B, nh, T, T)
scaled_scores = (q_final @ k_final.transpose(-2, -1)) / math.sqrt(head_dim)

# (We would apply the causal mask here)

attention_weights = F.softmax(scaled_scores, dim=-1)

# (B, nh, T, T) @ (B, nh, T, hd) -> (B, nh, T, hd)
output_per_head = attention_weights @ v_final

print("Shape of output from each head:", output_per_head.shape)
```

Output:

```
Shape of output from each head: torch.Size([1, 12, 4, 64])
```

We now have a `(64-dimensional)` output vector for each of our 4 tokens, from each of our 12 heads.

Step 3: Merging the Heads

The last step is to combine the insights from all 12 heads. We do this by reversing the reshape operation: we concatenate the heads back together into a single `C` -dimensional vector and then pass it through a final linear projection layer (`c_proj`).

```
# 1. Transpose and reshape to merge the heads back together
# (B, nh, T, hd) -> (B, T, nh, hd)
merged_output = output_per_head.transpose(1, 2).contiguous()
# The .contiguous() is needed because transpose can mess with memory layout.
# It creates a new tensor with the elements in the correct memory order.

# (B, T, nh, hd) -> (B, T, C)
merged_output = merged_output.view(B, T, C)

print("Shape of merged output:", merged_output.shape)

# 2. Pass through the final projection layer
c_proj = nn.Linear(C, C)
final_output = c_proj(merged_output)

print("Shape of final output:", final_output.shape)
```

Output:

```
Shape of merged output: torch.Size([1, 4, 768])
Shape of final output: torch.Size([1, 4, 768])
```

We have successfully returned to our original `(B, T, C)` shape. Each token's vector now contains the combined, context-aware information from all 12 attention heads.

Component	Shape Transformation	Purpose
Split Heads	`(B, T, C) -> (B, nh, T, hd)`	Prepare for parallel computation
Attention	`(B, nh, T, hd) -> (B, nh, T, hd)`	Each head computes context independently
Merge Heads	`(B, nh, T, hd) -> (B, T, C)`	Combine the insights from all heads
Final Projection	`(B, T, C) -> (B, T, C)`	Mix the combined information

Part 2: Encapsulating in the `nn.Module`

Let's now look at the full `CausalSelfAttention` class from `gpt2_min.py` and see how this logic is implemented.

The `__init__` Method

We add the `c_proj` layer and an assertion to ensure the dimensions are compatible.

```
class CausalSelfAttention(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        assert config.n_embd % config.n_head == 0
        self.n_head = config.n_head
        self.n_embd = config.n_embd
        # ... (c_attn and bias buffer from before)
        self.c_proj = nn.Linear(config.n_embd, config.n_embd, bias=True)
```

The `forward` Method

This is the full implementation, combining everything we have learned.

```
    def forward(self, x):
        B, T, C = x.size()

        # 1. Get QKV and split into heads
        qkv = self.c_attn(x)
        q, k, v = qkv.split(self.n_embd, dim=2)
        head_dim = C // self.n_head
        q = q.view(B, T, self.n_head, head_dim).transpose(1, 2)
        k = k.view(B, T, self.n_head, head_dim).transpose(1, 2)
        v = v.view(B, T, self.n_head, head_dim).transpose(1, 2)

        # 2. Run causal self-attention on each head
        att = (q @ k.transpose(-2, -1)) / math.sqrt(head_dim)
        att = att.masked_fill(self.bias[:, :, :T, :T] == 0, float("-inf"))
        att = F.softmax(att, dim=-1)
        y = att @ v

        # 3. Merge heads and project
        y = y.transpose(1, 2).contiguous().view(B, T, C)
        y = self.c_proj(y)

        return y
```

Every single line in this module should now be clear. We have built the most complex and important component of the Transformer from the ground up.

The rest of the model is surprisingly simple. We just need to add the "thinking" layer (the MLP) and then stack these blocks together.

Chapter 8: The "Thinking" Layer: The MLP

We have successfully built the `CausalSelfAttention` module. This is the "communication" layer of the Transformer. It allows tokens to gather and aggregate information from their context.

But gathering information is only half the battle. After each token has collected the context it needs, it needs time to "think" about it. It needs to process this new, context-rich information. This is the job of the **MLP**, or Multi-Layer Perceptron. It is also sometimes called a Position-wise Feed-Forward Network (FFN).

Let's look at the `gpt2_min.py` code we are about to build. It's refreshingly simple.

```
# gpt2_min.py (lines 56–65)
class MLP(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.fc = nn.Linear(config.n_embd, 4 * config.n_embd)
        self.proj = nn.Linear(4 * config.n_embd, config.n_embd)
        self.drop = nn.Dropout(config.dropout)

    def forward(self, x):
        x = self.fc(x)
        x = F.gelu(x)  # GPT-2 uses GELU
        x = self.drop(self.proj(x))
        return x
```

The Goal: Processing Information Locally

The MLP has a very simple but crucial role. While the attention layer allows tokens to interact with *each other*, the MLP processes the information for **each token independently**.

Imagine the attention layer was a group meeting where everyone shared ideas. The MLP is like each person going back to their desk to sit and think about what they just heard. They process the information on their own, without talking to anyone else, before the next group meeting (the next Transformer block).

This structure—communication followed by individual computation, repeated over many layers—is what gives the Transformer its power.

The Architecture: Expand and Contract

The MLP in a Transformer has a standard two-layer architecture:

1. **Expansion Layer (`fc`):** The first linear layer takes the input vector of size `n_embd` and projects it up to a much larger, intermediate dimension, typically `4 * n_embd` .
2. **Non-Linearity (`gelu`):** An activation function is applied. GPT-2 uses GELU (Gaussian Error Linear Unit), which is a smooth alternative to the more common ReLU. This is what allows the network to learn complex, non-linear functions.
3. **Contraction Layer (`proj`):** The second linear layer projects the large intermediate vector back down to the original `n_embd` dimension.
4. **Dropout (`drop`):** A dropout layer is applied for regularization to prevent overfitting.

Part 1: A Deeper Dive into `nn.Linear`

Before building the full MLP, let's demystify its core component: the `nn.Linear` layer. It's simpler than it sounds. At its heart, it's just a matrix multiplication followed by the addition of a bias vector.

The Math: `output = input @ W^T + b`

For each output element, the layer calculates a weighted sum of all input elements and adds a bias.

Let's see this with a tiny example. We'll project a vector of size 2 up to a vector of size 4.

```
import torch
import torch.nn as nn

C_in = 2
C_out = 4
linear_layer = nn.Linear(C_in, C_out)
```

What are the learnable parameters?

This layer has two sets of learnable parameters that are updated during training:

1. **Weights (`.weight`):** A matrix of shape `(C_out, C_in)` . For us, this is `(4, 2)` . Total weights: `4 * 2 = 8` .
2. **Biases (`.bias`):** A vector of shape `(C_out)` . For us, this is `(4)` . Total biases: `4` .

Let's manually set these parameters to simple integers to see the math clearly.

```
# Manually set the weights
linear_layer.weight.data = torch.tensor([
    [1., 0.],  # Weights for output element 0
    [-1., 0.], # Weights for output element 1
    [0., 2.],  # Weights for output element 2
    [0., -2.]  # Weights for output element 3
])

# Manually set the biases
linear_layer.bias.data = torch.tensor([1., 1., -1., -1.])
```

Now, let's pass a single input vector through it.

```
# Our input vector
input_vector = torch.tensor([0.5, -0.5])

# The forward pass
output_vector = linear_layer(input_vector)
```

Let's manually calculate the first output element to prove we understand the logic.

- `output[0] = (input[0] * weight[0,0]) + (input[1] * weight[0,1]) + bias[0]`
- `output[0] = (0.5 * 1.0) + (-0.5 * 0.0) + 1.0`
- `output[0] = 0.5 + 0.0 + 1.0 = 1.5`

Let's see the full result from PyTorch:

```
print("Input vector:", input_vector)
print("Output vector:", output_vector)
```

Output:

```
Input vector: tensor([ 0.5000, -0.5000])
Output vector: tensor([ 1.5000,  0.5000, -2.0000,  0.0000], grad_fn=<AddBackward0>)
```

The output matches our manual calculation for the first element. The `nn.Linear` layer simply performs this weighted sum for each of the 4 output elements. Now that this is clear, we can build the full MLP.

Part 2: The Full MLP Walkthrough with Numbers

Now that we understand how an `nn.Linear` layer works, let's trace a single token's vector through the entire MLP forward pass. The MLP acts on each token independently, so we only need to look at one vector to understand the whole process.

Our Setup:

- We'll use a tiny embedding dimension `C=2`.
- The MLP will expand this to an intermediate dimension of `4*C = 8`.
- Our input `x` will be the vector for a single token (`T=1`), in a batch of one (`B=1`).

```
# Our input vector for one token. Shape (B, T, C) -> (1, 1, 2)
x = torch.tensor([[[0.5, -0.5]]])
```

Step 1: The Expansion Layer (`fc`)
This is an `nn.Linear` layer that projects from `C=2` to `4*C=8`.

```
# Create the layer
fc = nn.Linear(2, 8)

# Manually set its weights and biases for a clear example
fc.weight.data = torch.randn(8, 2) * 2 # Scale up for more interesting GELU results
fc.bias.data = torch.ones(8) # Set all biases to 1

# --- Pass the input through the layer ---
x_expanded = fc(x)

print("--- After Expansion Layer ---")
print("Shape:", x_expanded.shape)
print("Values:\n", x_expanded.data.round(decimals=2))
```

Output:

```
--- After Expansion Layer ---
Shape: torch.Size([1, 1, 8])
Values:
 tensor([[[ 2.4000, -0.5000,  1.8800, -1.9100,  2.0800,  1.1600,  0.4100, -2.1200]]])
```

Our 2-dimensional vector has been successfully expanded to an 8-dimensional one.

Step 2: The GELU Activation

Next, we apply the non-linear GELU activation function. Intuitively, GELU is a smoother version of ReLU. It squashes negative values towards zero but allows a small amount of negative signal to pass through. Positive values are largely left unchanged.

Input	GELU(Input)
2.4	~2.39
1.0	~0.84
0.0	0.0
-0.5	~ -0.15
-2.0	~ -0.00

Let's apply it to our expanded vector:

```
import torch.nn.functional as F

# --- Apply GELU ---
x_activated = F.gelu(x_expanded)

print("\n--- After GELU Activation ---")
print("Shape:", x_activated.shape)
print("Values:\n", x_activated.data.round(decimals=2))
```

Output:

```
--- After GELU Activation ---
Shape: torch.Size([1, 1, 8])
Values:
 tensor([[[ 2.3900, -0.1500,  1.8700, -0.0100,  2.0600,  1.0300,  0.3100, -0.0000]]])
```

As expected, the large positive values (`2.40` , `1.88`) are almost untouched, while the large negative values (`-1.91` , `-2.12`) are squashed to nearly zero. This non-linear step is essential for the model to learn complex patterns.

Step 3: The Contraction Layer (`proj`)

Now, we project the 8-dimensional activated vector back down to our original `C=2` dimension.

```
# Create the layer
proj = nn.Linear(8, 2)

# Manually set its weights and biases
proj.weight.data = torch.randn(2, 8)
proj.bias.data = torch.zeros(2) # No bias for simplicity

# --- Pass the activated vector through the layer ---
x_projected = proj(x_activated)

print("\n--- After Contraction Layer ---")
print("Shape:", x_projected.shape)
print("Values:\n", x_projected.data.round(decimals=2))
```

Output:

```
--- After Contraction Layer ---
Shape: torch.Size([1, 1, 2])
Values:
 tensor([[[ 1.0900, -1.3800]]])
```

We are back to our original shape of `(1, 1, 2)`.

Step 4: Dropout

The final step in the `MLP` is dropout.

```
drop = nn.Dropout(0.1)
final_output = drop(x_projected)
```

During training, this layer would randomly set 10% of the elements in `x_projected` to zero. This is a regularization technique that helps prevent the model from becoming too reliant on any single feature.

During inference/evaluation (when we call `model.eval()`), the dropout layer does nothing and simply passes the data through unchanged. For our numerical example, we can assume it does nothing.

The Final Result

Our initial input vector `[[[0.5, -0.5]]]` has been transformed by the MLP into `[[[ 1.09, -1.38]]]`. This new vector, which has undergone a non-linear "thinking" process, is now ready for the next stage.

The key takeaway is that the MLP transforms the input vector while preserving its shape `(B, T, C)`. This is critical, as it allows us to add this output back to the original input (a "residual connection") and to stack multiple Transformer Blocks on top of each other.

We have now built both major components of our Transformer block: `CausalSelfAttention` (communication) and `MLP` (thinking). The final step is to assemble them into a complete `Block`.

Chapter 9: The Express Lane: Residual Connections

We have successfully built the two main computational engines of our model:

1. **`CausalSelfAttention`** : The "communication" layer where tokens exchange information.
2. **`MLP`** : The "thinking" layer where each token processes the information it has gathered.

Now, we need to assemble them into a robust, repeatable `Block` . To do this, we must introduce the architectural glue that makes deep learning possible. In this chapter, we will focus on the first and most important piece of that glue: the **Residual Connection**.

Let's look at the full `Block` class from `gpt2_min.py` . Our entire focus in this chapter is on the `+` operations in the `forward` method.

```python
# gpt2_min.py (lines 67-76)
class Block(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        # We will discuss these LayerNorm layers in the next chapter
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)

    def forward(self, x):
        """
        The forward pass of a single Transformer Block.
        """
        # --- This is our focus: the addition operation ---
        # The output of the attention layer is ADDED to the original input 'x'.
        x = x + self.attn(self.ln_1(x))

        # --- And this one too ---
        # The output of the MLP is ADDED to the result of the first step.
        x = x + self.mlp(self.ln_2(x))
        return x
```

This simple `x = x + ...` pattern, known as a residual or skip connection, is arguably one of the most significant innovations in the history of deep learning.

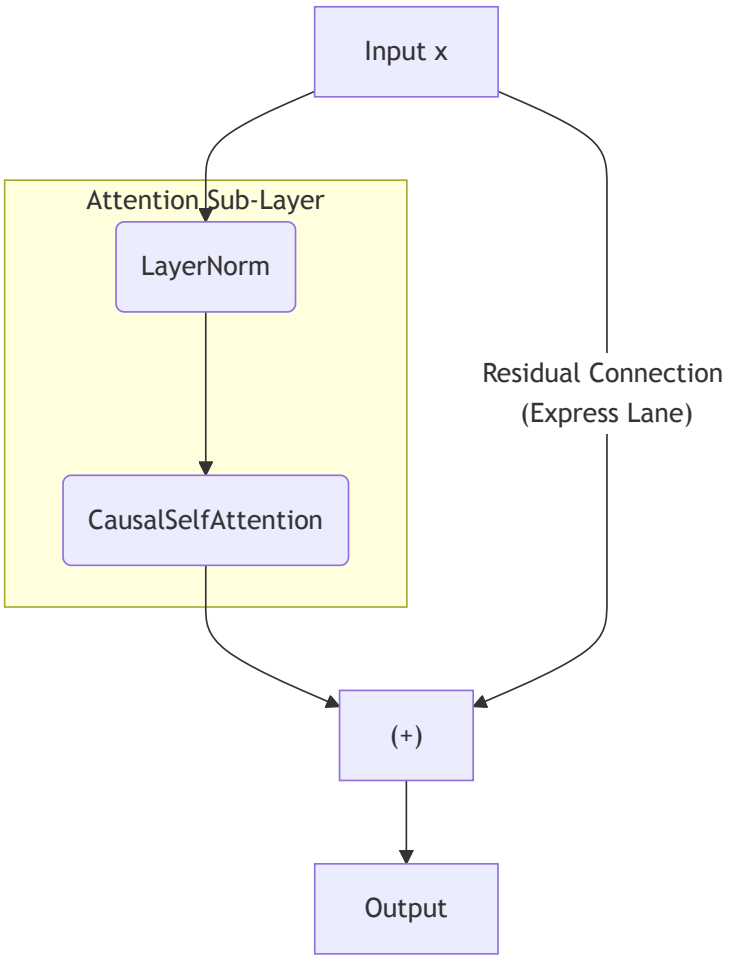
The Problem: Why Simple Stacking Fails (The Vanishing Gradient)

A natural first instinct when building a deep model is to just stack layers sequentially: `x -> layer1 -> layer2 -> layer3 -> ...` . However, when networks get very deep (e.g., more than a dozen layers), this simple approach often fails.

The reason is a phenomenon called the **vanishing gradient problem**. During training, the learning signal (the gradient) must travel backward from the final output all the way to the first layer's weights. With each step backward through a layer, this signal is multiplied by the layer's weights. In many cases, this causes the signal to shrink exponentially. By the time it reaches the early layers, it's so vanishingly small that those layers barely learn at all.

The Solution: The Residual "Express Lane"

The residual connection provides an elegant solution by creating a "shortcut" or an "express lane" for the data and, more importantly, for the gradient.



By adding the original input `x` directly to the output of the sub-layer (`self.attn(...)`), we create an uninterrupted highway. During backpropagation, the gradient can flow directly through this addition operator, completely bypassing the complex transformations inside the `attn` layer.

This changes the learning objective. The network no longer needs to learn the entire, complex transformation from scratch. Instead, the `attn` layer only needs to learn the *residual*—the difference, or "delta," that should be applied to the input.

Intuition: Imagine you're teaching a painter.

- **Without Residuals (Hard):** "Here is a blank canvas. Paint a masterpiece."
- **With Residuals (Easy):** "Here is the current painting (`x`). Just make these small, incremental adjustments (`attn(self.ln_1(x))`)."

The final result is `x + attn(self.ln_1(x))` . It is much easier for a network to learn how to make small, iterative adjustments than it is to learn the entire transformation at every single layer.

Walkthrough with Numbers

Let's see this in action. The operation is a simple element-wise addition. We'll focus on a single token for clarity (`B=1, T=1`) with an embedding dimension of `C=4` .

```
import torch

# Our input vector for a single token, 'x' at the start of the forward pass
x_initial = torch.tensor([[[0.2, 0.1, 0.3, 0.4]]])
print("Original input x:\n", x_initial)

# Let's pretend this is the output of `self.attn(self.ln_1(x))`.
# It represents the "change" or "adjustment" to be made.
attention_output = torch.tensor([[[0.1, -0.1, 0.2, -0.3]]])
print("\nOutput from the Attention sub-layer (the 'adjustment'):\n", attention_output)

# The residual connection is the first line of the forward pass: x = x + ...
x_after_attn = x_initial + attention_output
print("\nValue of x after the first residual connection:\n", x_after_attn)
```

Output:

```
Original input x:
 tensor([[[0.2000, 0.1000, 0.3000, 0.4000]]])

Output from the Attention sub-layer (the 'adjustment'):
 tensor([[[ 0.1000, -0.1000,  0.2000, -0.3000]]])

Value of x after the first residual connection:
 tensor([[[0.3000, 0.0000, 0.5000, 0.1000]]])
```

It's that simple. The output of the attention sub-layer is just an update to the original vector. The shape of the tensor remains unchanged, which is a critical property.

Step in `forward`	Operation	Input Shape	Output Shape	Meaning
1	`self.attn(self.ln_1(x))`	`(B, T, C)`	`(B, T, C)`	Calculate the update/residual
2	`x + ...`	`(B, T, C)`	`(B, T, C)`	Apply the update to the original input

We have now added the first piece of "glue" to our block. This express lane allows us to build much deeper and more powerful models. The next piece of glue we need is a stabilizer to keep the data flowing smoothly on this highway: Layer Normalization.

Chapter 10: Keeping it Stable: Layer Normalization

In the last chapter, we introduced the "express lane" of our Transformer Block: the residual connection. This allows us to build deep networks. However, a highway with no rules can lead to chaos. We need a "stabilizer" to ensure the data flowing through our network remains well-behaved. This is the role of **Layer Normalization**.

Let's look again at the full `Block` class from `gpt2_min.py`. Our focus is now on the `self.ln_1` and `self.ln_2` layers and where they are applied in the `forward` pass.

```
# gpt2_min.py (lines 67-76)
class Block(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        # --- We define the LayerNorm layers here ---
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)

    def forward(self, x):
        """
        The forward pass of a single Transformer Block.
        """
        # --- LayerNorm is applied BEFORE the sub-layer ---
        x = x + self.attn(self.ln_1(x))

        # --- And here again ---
        x = x + self.mlp(self.ln_2(x))
        return x
```

The Problem: Internal Covariate Shift

As data flows through a deep network, the distribution of the activations at each layer is constantly changing during training. The mean and variance of the inputs to a given layer can shift wildly from one training batch to the next. This phenomenon is called **internal covariate shift**.

This makes training very difficult. It's like trying to hit a moving target. Each layer has to constantly adapt to a new distribution of inputs from the layer before it, which can make the training process unstable and slow.

The Solution: Layer Normalization

Layer Normalization is a technique that forces the inputs to each sub-layer to have a consistent distribution. It acts as a stabilizer. For **each individual token's vector** in our `(B, T, C)` tensor, it performs the following steps *independently*:

1. Calculates the mean (μ) and variance (σ^2) across the `C` (embedding) dimension of that single vector.
2. Normalizes the vector: $\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$.
3. Applies learnable parameters: $y = \gamma \cdot \hat{x} + \beta$, where γ is a `gain` and β is a `bias`.

These learnable parameters are crucial. After forcing the distribution to a standard normal (mean 0, std 1), the model can then learn, via γ and β, to scale and shift this distribution to whatever is optimal for the next layer.

Walkthrough with Numbers

Let's trace the full Layer Normalization process for a single token's vector to see exactly what's happening. Our vector will have `C=4`.

Step 1: The Input Vector

Imagine this is the vector for one token after a residual connection. Its values have shifted away from a clean distribution during training.

```
import torch
import torch.nn as nn

# A sample vector for one token, with shape (B, T, C)
x_token = torch.tensor([[[0.3, -0.2, 0.8, 0.5]]])
print("Input to LayerNorm (x):\n", x_token)

# Let's calculate its current mean and standard deviation
mean = x_token.mean(dim=-1, keepdim=True)
std = x_token.std(dim=-1, keepdim=True)
print(f"\nMean of input: {mean.item():.2f}")
print(f"Std Dev of input: {std.item():.2f}")
```

Output:

```
Input to LayerNorm (x):
 tensor([[[ 0.3000, -0.2000,  0.8000,  0.5000]]])

Mean of input: 0.35
Std Dev of input: 0.41
```

The vector is not centered at zero and its values are not scaled to a standard deviation of one.

Step 2: Normalization (The Core $\hat{x}$ Calculation)

The first part of LayerNorm is to force the vector to have a mean of 0 and a standard deviation of 1. This is the normalization step, producing $\hat{x}$.

```
# A small value to prevent division by zero
epsilon = 1e-5

# Manually normalize
x_hat = (x_token - mean) / torch.sqrt(std**2 + epsilon)

print("Normalized vector (x_hat):\n", x_hat.data.round(decimals=2))
print(f"\nMean of x_hat: {x_hat.mean().item():.2f}")
print(f"Std Dev of x_hat: {x_hat.std().item():.2f}")
```

Output:

```
Normalized vector (x_hat):
 tensor([[[-0.1200, -1.3300,  1.0900,  0.3600]]])

Mean of x_hat: 0.00
Std Dev of x_hat: 1.00
```

Perfect. This is the core stabilizing operation.

Step 3: Applying the Learnable Parameters (γ and β)

The final step is to apply the learnable `gain` (γ) and `bias` (β). These parameters are created automatically when you instantiate `nn.LayerNorm`. Initially, γ is a vector of all ones and β is a vector of all zeros.

```
C = 4
ln = nn.LayerNorm(C)
print("--- Initial Parameters ---")
print(f"LayerNorm.weight (gamma) initial:\n {ln.weight.data}")
print(f"LayerNorm.bias (beta) initial:\n {ln.bias.data}")
```

Output:

```
--- Initial Parameters ---
LayerNorm.weight (gamma) initial:
 tensor([1., 1., 1., 1.])
LayerNorm.bias (beta) initial:
 tensor([0., 0., 0., 0.])
```

Let's pretend that during training, the model learned that a different scaling and shifting is optimal. We can set these parameters manually to see their effect.

```
gamma = torch.tensor([1.5, 1.0, 1.0, 1.0])
beta = torch.tensor([0.5, 0.0, 0.0, 0.0])

# --- Manually apply gamma and beta ---
y = gamma * x_hat + beta

print("\n--- After Applying Learned Gamma and Beta ---")
print("Final output vector (y):\n", y.data.round(decimals=2))
print(f"\nMean of y: {y.mean().item():.2f}")
print(f"Std Dev of y: {y.std().item():.2f}")
```

Output:

```
--- After Applying Learned Gamma and Beta ---
Final output vector (y):
 tensor([[[ 0.3200, -1.3300,  1.0900,  0.3600]]])

Mean of y: 0.11
Std Dev of y: 0.94
```

The final `nn.LayerNorm` module performs all these steps in one efficient call. The model has used the learnable γ and β to find the most useful distribution for the next layer.

Pre-Norm vs. Post-Norm

The exact placement of Layer Normalization relative to the residual connection is an important architectural choice.

Feature	Pre-Norm (GPT-2 style)	Post-Norm (Original Transformer style)
Equation	`x + Sublayer( LayerNorm(x) )`	`LayerNorm( x + Sublayer(x) )`
Stability	Generally leads to **more stable training** for very deep networks. Easier to train from scratch without warm-up schedules.	Can be harder to train for very deep models. Often requires learning rate warm-up.
Example Code	`x = x + self.attn(self.ln_1(x))`	`x = self.ln_1(x + self.attn(x))`

Why GPT-2 uses Pre-Norm:
Pre-Norm helps prevent the internal activations from growing too large. Since the input to each sub-layer (Attention or MLP) is always normalized, it keeps the magnitudes of values in check, which directly contributes to a more stable training process, especially for very deep models like GPT.

With residual connections providing the "express lane" and layer normalization acting as the "stabilizer," we now have all the necessary components to assemble a complete, robust, and trainable Transformer Block. In the next chapter, we will put it all together.

Chapter 11: Assembling One Complete Transformer Block

This chapter is the grand payoff for all the components we've built in Part 3. We will now take our `CausalSelfAttention` and `MLP` modules and assemble them, using the architectural glue of `Residual Connections` and `Layer Normalization`, into one complete, powerful, and stackable `Block`.

This `Block` is the fundamental repeating unit of the entire GPT-2 model. First, let's look at our final destination: the code for the `Block` itself, and the code that stacks it to build the full model.

```
# The Lego Brick: One complete Transformer Block
# We will assemble this in this chapter.
class Block(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.ln_1 = nn.LayerNorm(config.n_embd)
        self.attn = CausalSelfAttention(config)
        self.ln_2 = nn.LayerNorm(config.n_embd)
        self.mlp = MLP(config)

    def forward(self, x):
        x = x + self.attn(self.ln_1(x))
        x = x + self.mlp(self.ln_2(x))
        return x

# Stacking the Bricks: How the Block is used in the full GPT2 model
# We will build this in the next chapter.
class GPT2(nn.Module):
    def __init__(self, config: GPTConfig):
        # ... (other layers)
        self.h = nn.ModuleList([Block(config) for _ in range(config.n_layer)])
        # ... (other layers)
```

The Blueprint of a `Block` (`__init__`) {#the-blueprint-of-a-block-init}

The constructor of our `Block` is simple because it's just an assembly of the complex parts we've already built.

```
def __init__(self, config: GPTConfig):
    super().__init__()
    self.ln_1 = nn.LayerNorm(config.n_embd)
    self.attn = CausalSelfAttention(config)
    self.ln_2 = nn.LayerNorm(config.n_embd)
    self.mlp = MLP(config)
```

- `self.ln_1` : The first "stabilizer" (LayerNorm), applied just before the attention layer.
- `self.attn` : The "communication" layer (`CausalSelfAttention`), where tokens exchange information.
- `self.ln_2` : The second "stabilizer," applied just before the MLP layer.
- `self.mlp` : The "thinking" layer (`MLP`), where each token processes the information it has gathered.

The Data's Journey Through the Block (`forward`)

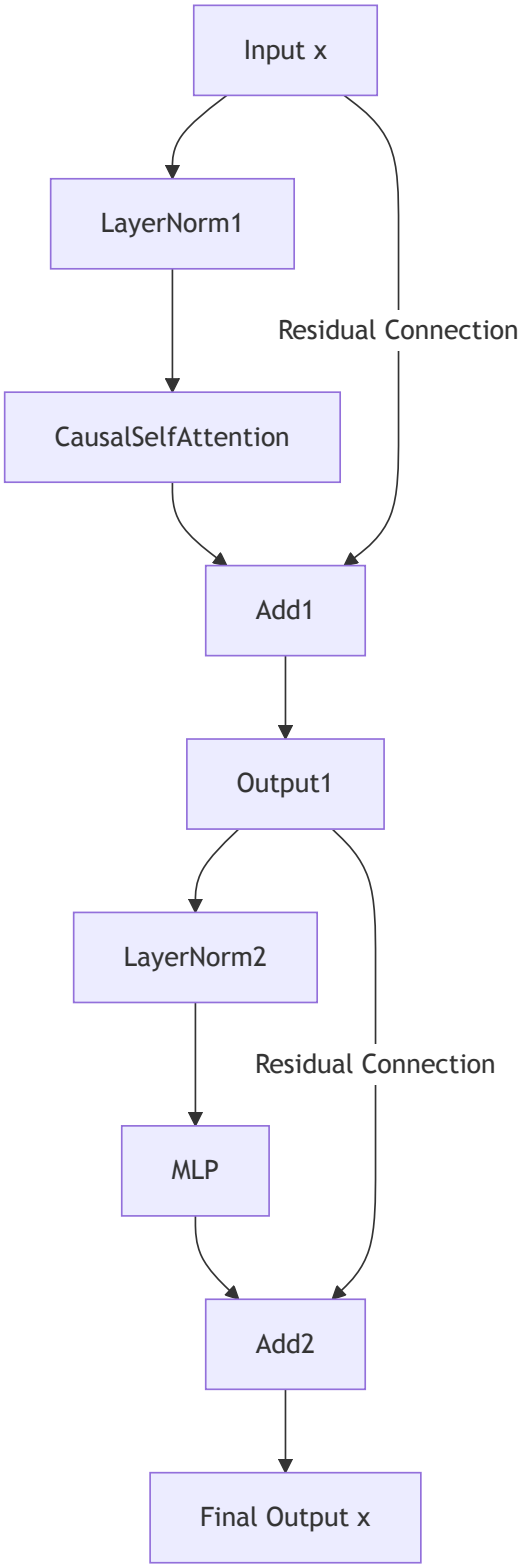
The `forward` method orchestrates the flow of data through these components, following the "Pre-Norm" architecture.

```
def forward(self, x):
    # First sub-layer: Attention
    x = x + self.attn(self.ln_1(x))

    # Second sub-layer: MLP
    x = x + self.mlp(self.ln_2(x))

    return x
```

The logic for each sub-layer is identical and beautifully simple: **Normalize, Process, Add.**

1. **Normalize:** The input `x` is first passed through `self.ln_1`.
2. **Process:** The stabilized output is then passed through the `self.attn` layer.
3. **Add:** The output of the attention layer is added back to the *original, unmodified input* `x` via the residual connection.

This process is then repeated for the MLP sub-layer.



The most critical property of this entire block is that the shape of the output tensor is **identical to the shape of the input tensor (`B, T, C`)**. This is what makes the block "stackable."

Stacking Blocks for Depth: From a Single Meeting to a Symposium

This "stackability" is the key to the Transformer's power. A single `Block` can only perform one round of "communication" (attention) and "thinking" (MLP). This is like a single project meeting. The team gets together, shares information, and then goes back to their desks to process it. It's a good first step, but complex problems require more than one meeting.

To build a truly deep understanding of language, we need to hold a series of these meetings—a symposium.

- **Meeting 1 (Block 0):** The team starts with the raw project proposal (the token embeddings). They discuss it, and each member leaves with a refined, first-level understanding of the project.
- **Meeting 2 (Block 1):** The team reconvenes. They don't start from the raw proposal again. Instead, they start with the *refined understanding they gained from the first meeting*. This allows them to discuss higher-level concepts and strategies.
- **Meeting 12 (Block 11):** After a long series of such meetings, the team's understanding is incredibly deep and nuanced. They've moved from basic syntax to deep semantic meaning.

This is exactly what stacking Transformer Blocks does. The output of one block becomes the input for the next, allowing the model to build a hierarchical understanding of the text.

Distinguishing Depth from Width
Now we can clearly distinguish between what happens *in* a meeting versus the *series* of meetings.

Concept	What it Does	Analogy	Why?
Width (Multi-Head)	Parallel processing within a layer.	A committee of 12 specialists **in one meeting**.	To analyze input from many perspectives at the same level of abstraction.
Depth (Multi-Layer)	Sequential processing across layers.	**A series of 12 meetings**, each refining the last.	To build a hierarchical understanding, from simple syntax to abstract semantics.

Implementing Depth with `nn.ModuleList`

Now, let's look at the code that implements this series of meetings. This line is from the main `GPT2` model's `__init__` method.

```
# From the GPT2 class __init__ method
self.h = nn.ModuleList([Block(config) for _ in range(config.n_layer)])
```

- `[...] for _ in range(config.n_layer)` : This creates `n_layer` (e.g., 12) separate instances of our `Block` .
- `nn.ModuleList([...])` : This special PyTorch container registers all 12 `Block` s as part of our model, so PyTorch can track all their parameters.

Are the weights shared between blocks?
No, they are not. This is the crucial implementation detail that enables the symposium analogy. When the code calls `Block(config)` twelve separate times, it creates twelve brand-new `Block` objects. Each one has its own unique set of weights for its `attn` and `mlp` layers.

This is essential. The "skills" needed for the first meeting (processing raw embeddings) are different from the skills needed for the last meeting (performing final refinements on an abstract representation). By giving each block its own set of weights, we allow each layer in the stack to specialize in its particular stage of the processing pipeline.

We have now officially finished building our fundamental Lego brick and we understand the strategy for stacking them. We are ready to move on to construct the final model architecture.

Chapter 12: Stacking the Blocks: The Full GPT Model

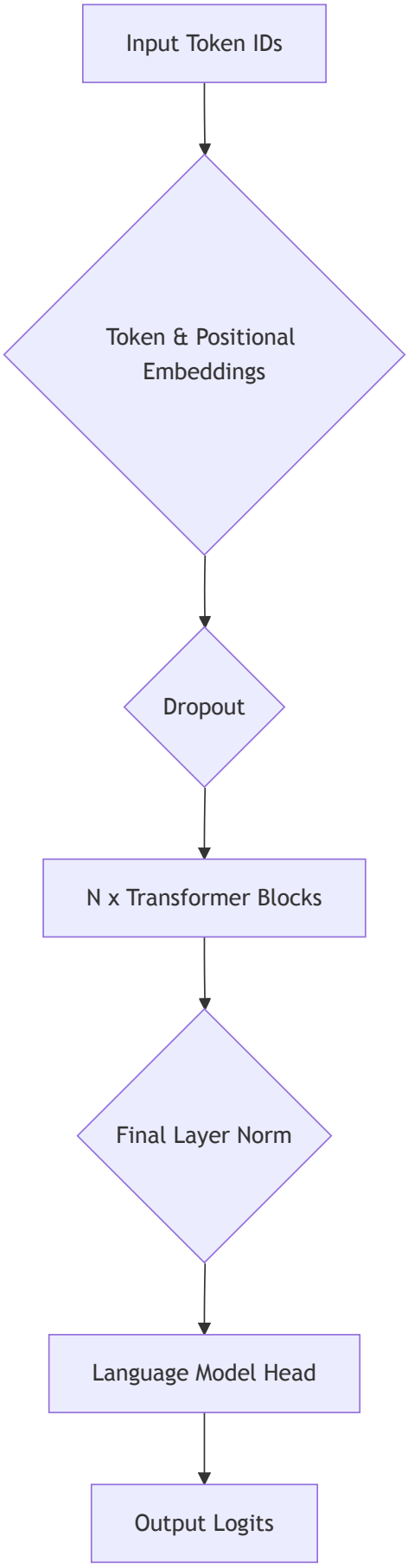
We are now at the top of the mountain. We have built every single custom component required for our GPT model. All that's left is to assemble them in the correct order, following the blueprint laid out in the `GPT2` class. This chapter will feel like a victory lap, as you will recognize every single line.

Let's start by looking at the `__init__` method of our final model. This is the constructor that defines and organizes all the layers we've discussed.

```
# gpt2_min.py (lines 78–90)
class GPT2(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        self.config = config

        # --- Part 1: The Input Layers ---
        self.wte = nn.Embedding(config.vocab_size, config.n_embd)  # token embeddings
        self.wpe = nn.Embedding(config.block_size, config.n_embd)  # positional embeddings
        self.drop = nn.Dropout(config.dropout)

        # --- Part 2: The Core Processing Layers ---
        self.h = nn.ModuleList([Block(config) for _ in range(config.n_layer)])

        # --- Part 3: The Output Layers ---
        self.ln_f = nn.LayerNorm(config.n_embd)

        # ... (Language Model Head will be in the next chapter)
```

The Architecture of the Full Model

This `__init__` method perfectly follows the high-level architecture we set out to build from the very beginning.



Let's walk through the `__init__` method section by section.

Part 1: The Input Layers

```
self.wte = nn.Embedding(config.vocab_size, config.n_embd)
self.wpe = nn.Embedding(config.block_size, config.n_embd)
self.drop = nn.Dropout(config.dropout)
```

This is the "entry point" of our model. It handles the initial conversion of raw token IDs into meaningful vectors.

- `self.wte` (Word Token Embedding): The learnable dictionary that maps a token's ID to its initial vector representation (Chapter 2).
- `self.wpe` (Word Position Embedding): The learnable lookup table that provides a vector representing each token's position in the sequence (Chapter 3).
- `self.drop` : A dropout layer applied to the sum of these embeddings. This is a regularization technique that helps prevent the model from overfitting by randomly setting some of the input features to zero during training.

Part 2: The Core Processing Layers

```
self.h = nn.ModuleList([Block(config) for _ in range(config.n_layer)])
```

This is the heart of the model—the "deep" part of deep learning. As we discussed in the last chapter, this line creates `n_layer` (e.g., 12) independent `Block` instances and registers them in an `nn.ModuleList` . The data will flow through these blocks sequentially, becoming more contextually refined at each step.

Part 3: The Final Output Layer

```
self.ln_f = nn.LayerNorm(config.n_embd)
```

- `self.ln_f` (Final Layer Norm): After the data has passed through the entire stack of Transformer blocks, a final layer normalization is applied. This provides one last stabilization step, ensuring the output vectors are well-behaved before they are passed to the final prediction layer.

We have now defined almost all the structural components of our model. We've built the entrance, the main processing tower, and the final stabilization stage.

The only piece missing is the most important one for a language model: the layer that actually makes the prediction. How do we go from these highly processed vectors back to a probability distribution over our entire vocabulary? That is the job of the Language Model Head, which we will build in the very next chapter.

Chapter 13: The Grand Finale: The Language Model Head

We have reached the final architectural component of our GPT model. Our data has been converted into embeddings, processed through a deep stack of Transformer blocks, and stabilized by a final layer norm. We are left with a tensor of highly context-aware vectors, one for each token in our sequence.

The Problem: Our final processed tensor has a shape of `(B, T, C)` , for example `(1, 4, 768)` . This means for each of the 4 tokens in our input "A crane ate fish," we have a rich, 768-dimensional vector. This is an internal representation, not a prediction. How do we use these vectors to predict the single *next* word?

A natural assumption would be to take the vector for the very last token ("fish") and use it to make one prediction. But the Transformer architecture does something far more clever and efficient. Let's see how by examining the final piece of our `GPT2` model's `__init__` method.

```
# gpt2_min.py (lines 78–94, abbreviated)
class GPT2(nn.Module):
    def __init__(self, config: GPTConfig):
        super().__init__()
        # ... (wte, wpe, drop)
        self.h = nn.ModuleList([Block(config) for _ in range(config.n_layer)])
        self.ln_f = nn.LayerNorm(config.n_embd)

        # --- The final projection layer ---
        self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
        self.lm_head.weight = self.wte.weight
```

The `lm_head` : A Parallel Prediction Layer

The `lm_head` is a simple `nn.Linear` layer that projects from our internal vector space (`C` dimensions) into the vocabulary space (`vocab_size` dimensions). The key insight is that this projection is applied **independently and in parallel to every single token's vector** along the `T` dimension.

Instead of making one prediction, it makes `T` predictions. This results in the following shape transformation:

Variable	Input Shape `(B, T, ...)`	Output Shape `(B, T, ...)`	Meaning of Output
`logits`	`(B, T, C)`	`(B, T, vocab_size)`	A raw score for every possible next token, for **each** of the `T` input positions.

This reveals the "twist": the output `logits` tensor doesn't contain one prediction, it contains `T` predictions. This brings us to a new, crucial question.

"Why make `T` predictions? Isn't that wasteful?"

This is a brilliant design choice that makes GPT models both efficient to train and effective at generation. The purpose of these parallel predictions depends entirely on the task at hand.

1. For Efficient Training:
During training, we want to teach the model to predict the next word at *every position* in the sequence, all at once.

- **Input:** "A crane ate fish" (`T=4`)
- **Targets:** The model should learn:
  - Given "A", predict "crane".
  - Given "A crane", predict "ate".
  - Given "A crane ate", predict "fish".

The `logits` tensor gives us all the predictions we need in a single forward pass:

- `logits[:, 0, :]` is the model's prediction based on the context "A". We will compare this to the target "crane".

- `logits[:, 1, :]` is the prediction based on the context "A crane". We compare this to "ate".
- `logits[:, 2, :]` is the prediction based on the context "A crane ate". We compare this to "fish".

(Thanks to our causal mask, we know the prediction at position `t` only depends on tokens `0` to `t`). This parallel approach is incredibly efficient for training.

2. For Generation (Inference):
When we want to generate new text, the observation is correct: we are seemingly "wasteful."

- **Input:** A prompt, e.g., "A crane ate" (`T=3`)
- The model produces a `logits` tensor of shape `(1, 3, vocab_size)`.
- We **ignore** the predictions from the first two positions (`logits[:, 0, :]` and `logits[:, 1, :]`).
- We **only use** the prediction from the final position, `logits[:, -1, :]`, to sample the next word.

This might seem inefficient, but this design is a trade-off. The architecture is heavily optimized for the massively parallel computations needed for training. During inference, we leverage this same powerful parallel architecture, even if we only need the result from the final time step. Many modern inference optimizations focus on avoiding the re-computation of these "wasted" intermediate steps.

The Final Trick: Weight Tying

Let's look closely at the last two lines in our model's `__init__` method, which define the Language Model Head:

```
# From the GPT2 class __init__ method
self.lm_head = nn.Linear(config.n_embd, config.vocab_size, bias=False)
self.lm_head.weight = self.wte.weight
```

The first line is straightforward: it creates a standard linear layer. The second line, however, is a simple but profound optimization known as **weight tying**.

What is the code actually doing?
Instead of allowing the `lm_head` to have its own, randomly initialized weight matrix that would be learned during training, this line of code literally throws that matrix away. It then assigns the `.weight` attribute of the `lm_head` to be a *reference* to the `.weight` attribute of the `wte` (our token embedding layer).

From this point on, these two layers **share the exact same weight matrix**. When backpropagation updates the weights of the `lm_head`, it is simultaneously updating the weights of the `wte`, and vice-versa. They are not just two matrices that happen to be identical; they are, in memory, the very same object.

Why does this make sense?
Let's analyze the roles and shapes of the two weight matrices:

Layer	Attribute	Shape `(rows, cols)`	Role
`wte`	`self.wte.weight`	`(vocab_size, n_embd)`	To convert a token ID (row index) into an `n_embd`-dimensional vector.
`lm_head`	`self.lm_head.weight`	`(vocab_size, n_embd)`	To convert an `n_embd`-dimensional vector into a score for each of the `vocab_size` tokens.

They have the exact same shape! Let's think about their functions intuitively:

- The **Token Embedding** matrix (`wte.weight`) can be thought of as an "ID-to-meaning" lookup table. Row `i` of this matrix is the learned vector that represents the "meaning" of the `i` -th word in the vocabulary.
- The **Language Model Head** matrix (`lm_head.weight`) can be thought of as a "meaning-to-ID" lookup table. When we multiply our final processed vector with this matrix, we are essentially comparing our vector's "meaning" against the "meaning" vector of every word in the vocabulary (each row of the matrix). The words whose meaning vectors align best get the highest scores (logits).

The core insight of weight tying is that these two operations—mapping from an ID to a meaning, and mapping from a meaning back to an ID—should be symmetric. The representation of a word should be the same whether it's an input or an output. By forcing them to share the same weight matrix, we build this strong and logical assumption directly into the model's architecture.

The Benefits:

1. **Massive Parameter Reduction:** The `lm_head` is one of the largest layers in the model. For GPT-2 small, this matrix has `50257 * 768 ≈ 38.5 million` parameters. By tying the weights, we eliminate the need to store and train a second, separate matrix of this size.
2. **Improved Performance:** This technique often acts as a powerful form of regularization. By enforcing a sensible architectural constraint, it can prevent overfitting and lead to better model performance.

We have now, finally, built a complete, end-to-end GPT model architecture, complete with this elegant optimization. The next step is to look at the full `forward` pass to see how the loss is calculated for training.

Chapter 14: Training the Model: The Forward Pass and Loss Calculation

We have successfully built the complete `GPT2` model architecture. We have an uneducated machine, full of randomly initialized weights. Now, we must teach it. This chapter focuses on the `forward` pass—the journey of data through our model to produce a single, crucial number: the **loss**. This loss value quantifies how "wrong" the model's predictions are and is the signal used to update all the weights via backpropagation.

Let's look at the `forward` method from `gpt2_min.py` that we will now fully understand.

```
# gpt2_min.py (lines 104–121)
class GPT2(nn.Module):
    # ... (__init__ method from previous chapters) ...

    def forward(self, idx, targets=None):
        B, T = idx.size()
        assert T <= self.config.block_size, "Sequence length exceeds block size."

        pos = torch.arange(0, T, dtype=torch.long, device=idx.device).unsqueeze(0)

        x = self.wte(idx) + self.wpe(pos)
        x = self.drop(x)
        for block in self.h:
            x = block(x)
        x = self.ln_f(x)
        logits = self.lm_head(x)

        loss = None
        if targets is not None:
            loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))

        return logits, loss
```

The Goal of Training: Next-Token Prediction

The training process for GPT is based on a simple principle: **given a sequence of words, predict the very next word.** To do this, we need to prepare our training data—a massive corpus of text—into `(input, target)` pairs using the **simple chunking** method we discussed. For each chunk, the `idx` is the input, and the `targets` are the input shifted by one position.

A Walkthrough of the `forward` Method

Let's trace the data flow step-by-step for one of these chunks.

1. **Get Embeddings:**

   ```
   pos = torch.arange(0, T, dtype=torch.long, device=idx.device).unsqueeze(0)
   x = self.wte(idx) + self.wpe(pos)
   ```

   This is exactly what we built in Chapters 2 and 3. We create token embeddings and add positional embeddings to get our initial `(B, T, C)` tensor.
2. **Process through Blocks:**

   ```
   x = self.drop(x)
   for block in self.h:
       x = block(x)
   ```

   The initial tensor is passed through a dropout layer and then sequentially through every `Block` in our `self.h` ModuleList. With each pass through a block, the token vectors become more and more context-aware.
3. **Get Logits:**

```
x = self.ln_f(x)
logits = self.lm_head(x)
```

The output from the final block is stabilized with a LayerNorm, and then projected by the `lm_head` to get our final `logits` tensor of shape `(B, T, vocab_size)`. This tensor contains `T` predictions, one for each position in our input sequence.

Calculating the Loss: One Number to Rule Them All

We now have our `logits` (the model's `T` predictions) and our `targets` (the `T` correct answers). The final step is to compare them to get a single loss value. This is done with `F.cross_entropy`.

```
loss = F.cross_entropy(logits.view(-1, logits.size(-1)), targets.view(-1))
```

This line looks dense, but it's doing something very methodical. `F.cross_entropy` in PyTorch expects its input in a specific 2D format, so we first need to reshape our tensors.

- `logits` has a shape of `(B, T, vocab_size)`.
- `targets` has a shape of `(B, T)`.
- The `.view(-1, ...)` function is PyTorch's way of reshaping. We are squashing the Batch and Time dimensions together.

Shape Transformation for the Loss Function

Variable	Original Shape	Reshaped with `.view()`	Purpose
`logits`	`(B, T, vocab_size)`	`(B*T, vocab_size)`	A 2D tensor where each row is a prediction for one position.
`targets`	`(B, T)`	`(B*T)`	A 1D tensor of the correct token IDs, aligned with the predictions.

How does `cross_entropy` average the loss?
A crucial point is that `F.cross_entropy` calculates the loss for *each* of the `B*T` predictions individually and then **averages them** to produce a single, final scalar `loss` value.

It does not add them up; it takes the mean. This is important because it keeps the loss on a consistent scale regardless of the batch size or sequence length.

The Full Training Loop: From a Text Stream to a Weight Update

Let's put it all together. A "training step" consists of processing one "batch" of data. To understand what a batch is, we first need to see how our raw text data is prepared.

Imagine our training data is one long stream of token IDs. For this example, let's say our `block_size` (or `T`) is `4`.

Step 1: Preparing the Data Stream
First, we lay out our continuous stream of text tokens.

- **Text Stream (Token IDs):** `[5, 12, 8, 21, 6, 33, 9, 4, 15, 7, 2, ...]`

Step 2: Creating `(input, target)` Pairs
From this stream, we create our `(idx, targets)` pairs. The `targets` are simply the `idx` shifted one position to the right.

- **Sample 1:** `idx` = `[5,12,8,21]`, `targets` = `[12,8,21,6]`
- **Sample 2:** `idx` = `[6,33,9,4]`, `targets` = `[33,9,4,15]`

Step 3: Forming a Batch
A **batch** is a stack of these training samples. We process multiple samples at once to make training on GPUs highly efficient. Let's create a batch with a batch size of `B=2`.

- **Input `idx` (shape `(2, 4)`):**

```
[[ 5, 12,  8, 21],  <-- Sample 1
 [ 6, 33,  9,  4]]   <-- Sample 2
```

- **Target `targets` (shape `(2, 4)`):**

```
[[12,  8, 21,  6],  <-- Targets for Sample 1
 [33,  9,  4, 15]]   <-- Targets for Sample 2
```

This batch is the single unit of data that will be processed in one training step.

Step 4: The Training Step - A Detailed Walkthrough

Now, let's trace this batch through the five stages of a single training step.

Stage	What Happens	Detailed Breakdown
1. Forward Pass	The batch (`idx`) is fed into the model's `forward` method.	The model processes both samples in parallel. This involves embeddings, 12 `Block`s of attention and MLPs, etc. The final output is a `logits` tensor of shape `(2, 4, vocab_size)`.
2. Loss Computation (Part A - Reshaping)	We prepare `logits` and `targets` for the loss function.	`logits.view(-1, vocab_size)` reshapes `logits` to `(8, vocab_size)`. `targets.view(-1)` reshapes `targets` to a 1D tensor of shape `(8)`, which looks like: `[12, 8, 21, 6, 33, 9, 4, 15]`.
3. Loss Computation (Part B - Cross-Entropy)	`F.cross_entropy` calculates the loss for each of the 8 prediction/target pairs.	Let's imagine the individual (negative log likelihood) losses are: • For `(pred_0, target_0=12)`: loss = 2.5 • For `(pred_1, target_1=8)`: loss = 3.1 • For `(pred_2, target_2=21)`: loss = 1.9 • For `(pred_3, target_3=6)`: loss = 4.2 • For `(pred_4, target_4=33)`: loss = 2.8 • For `(pred_5, target_5=9)`: loss = 3.5 • For `(pred_6, target_6=4)`: loss = 2.2 • For `(pred_7, target_7=15)`: loss = 3.8

Stage	What Happens	Detailed Breakdown

4. Loss Computation (Part C - Averaging)	`F.cross_entropy` **averages** these individual losses into one final number.	`final_loss = (2.5 + 3.1 + 1.9 + 4.2 + 2.8 + 3.5 + 2.2 + 3.8) / 8`
		`final_loss = 24.0 / 8 = 3.0`
		Our single, scalar `loss` value is `3.0`.

5. Backpropagation	**One** backpropagation is performed for the entire batch.	`loss.backward()` is called on the single scalar value `3.0`. PyTorch calculates the gradient of this *average loss* with respect to every single parameter in the entire model.
6. Weight Update	The optimizer takes one step.	`optimizer.step()` uses these gradients to update all the model's weights, nudging them in a direction that would have lowered that average loss of `3.0`.

This entire 6-stage process is one training step. It is repeated millions of times with different batches of data. The key takeaway is that for each batch, there is **only one backward pass and one weight update**, driven by the *average* performance across all token predictions in that batch.

We have now closed the loop: from a raw stream of text to a trained model. All that's left is the most exciting part: seeing how this trained model can be used to generate new text.

Chapter 15: Bringing It to Life: Autoregressive Generation

We have built the entire GPT architecture and understood how it's trained. Now, we unlock its true purpose: generating new, coherent text. The process is called **autoregressive generation**, which sounds complex but is based on a beautifully simple loop.

Let's look at the `generate` method from `gpt2_min.py`. This is the code that performs the magic.

```python
# gpt2_min.py (lines 123–143)
class GPT2(nn.Module):
    # ... (__init__ and forward methods) ...

    @torch.no_grad()
    def generate(self, idx, max_new_tokens=50, temperature=1.0, top_k=None):
        for _ in range(max_new_tokens):
            idx_cond = idx[:, -self.config.block_size:]
            logits, _ = self(idx_cond)
            logits = logits[:, -1, :] / max(temperature, 1e-8)

            if top_k is not None:
                v, _ = torch.topk(logits, min(top_k, logits.size(-1)))
                thresh = v[:, -1].unsqueeze(-1)
                logits = torch.where(logits < thresh, torch.full_like(logits, -float("inf")), logits)

            probs = F.softmax(logits, dim=-1)
            next_token = torch.multinomial(probs, num_samples=1)
            idx = torch.cat((idx, next_token), dim=1)
        return idx
```

The Core Idea: The Generation Loop

The process of generation is a one-token-at-a-time loop:

1. **PREDICT:** Feed the current sequence of tokens into the model to get the logits for the next token.
2. **SAMPLE:** Convert the logits into probabilities and sample one token from that distribution. This will be our newly generated token.
3. **APPEND:** Add the newly sampled token to the end of our sequence.
4. **REPEAT:** Go back to step 1 with the new, longer sequence.

Let's walk through this loop with a concrete example. Imagine we give the model the starting prompt "A crane".

- **Initial `idx`:** `[5, 12]`

Iteration 1:

1. **PREDICT:** We feed `idx = [5, 12]` into `model.forward()`. It produces `logits` of shape `(1, 2, vocab_size)`. We only care about the prediction for the *last* token, so we select `logits[:, -1, :]`. This is a vector of scores for the word to follow "crane".
2. **SAMPLE:** We apply softmax to these logits to get probabilities. Let's say the model gives "ate" a 40% probability, "lifted" a 35% probability, and so on. We sample from this distribution and get the token for "ate" (ID `8`).
3. **APPEND:** We concatenate this new token to our sequence. `idx` is now `[5, 12, 8]`.

Iteration 2:

1. **PREDICT:** We feed the *new* `idx = [5, 12, 8]` into the model. It gives us the logits for the word to follow "ate".
2. **SAMPLE:** We convert to probabilities. The model, having seen "A crane ate", now gives a very high probability to "fish". We sample and get the token for "fish" (ID `21`).
3. **APPEND:** `idx` becomes `[5, 12, 8, 21]`.

This loop continues until we reach the desired `max_new_tokens`.

A Deeper Look at the `generate` Code

Now let's connect this loop to the actual code.

1. **`@torch.no_grad()`**: This is a PyTorch decorator that tells the model not to calculate gradients. It's a crucial optimization for inference, as it saves a lot of memory and computation.
2. **Context Cropping:**

   ```
   idx_cond = idx[:, -self.config.block_size:]
   ```

   Our model has a fixed context window (`block_size`). If the sequence `idx` becomes longer than this, we must crop it to only include the last `block_size` tokens. This is the "memory" of the model.
3. **Getting the Final Logits:**

   ```
   logits, _ = self(idx_cond)
   logits = logits[:, -1, :] / max(temperature, 1e-8)
   ```

   - `self(idx_cond)` is just calling our `forward` method.
   - `logits[:, -1, :]` is the key step where we **throw away all predictions except the very last one**.
   - `/ temperature`: This is a knob to control the "creativity" of the output. We'll discuss it below.
4. **Sampling the Next Token:**

   ```
   probs = F.softmax(logits, dim=-1)
   next_token = torch.multinomial(probs, num_samples=1)
   ```

   - `F.softmax` converts our final logits into a probability distribution.
   - `torch.multinomial` performs the sampling. It takes the probabilities and randomly picks one token, where tokens with higher probabilities are more likely to be chosen.
5. **Appending:**

   ```
   idx = torch.cat((idx, next_token), dim=1)
   ```

   The newly sampled `next_token` is concatenated to the end of our `idx` sequence, preparing it for the next iteration of the loop.

Controlling the Magic: `temperature` and `top_k`

Randomly sampling from the full probability distribution can sometimes lead to strange or nonsensical words being chosen. We have two knobs to control this:

Parameter	What it Does	Effect
`temperature`	Rescales the logits before softmax. `logits / temp`.	**Low Temp (<1.0):** Makes the distribution "peakier". High-probability tokens become even more likely. The model becomes more confident and deterministic, but also more repetitive. **High Temp (>1.0):** Flattens the distribution. Low-probability tokens become more likely. The model becomes more random and creative, but also more prone to errors. **Note:** The code clamps temperature with `max(temperature, 1e-8)` to avoid

Parameter	What it Does	Effect
		division by zero, so `temp→0` approaches greedy sampling without exactly reaching it.
`top_k`	Truncates the distribution. Considers only the `k` most likely tokens.	If `top_k=50`, the model calculates the probabilities for all 50257 tokens, but then throws away all but the 50 most likely ones. It then re-normalizes the probabilities among just those 50 and samples from that smaller set. This effectively prevents very rare or nonsensical words from ever being chosen.

Conclusion: The Transformer Has Clicked

We have reached the end of our 90-minute journey. Let's take a moment to look back at what we've accomplished.

We started with a file, `gpt2_min.py`, that seemed like a dense, magical black box. We made a promise: to take that box apart, piece by piece, until the magic dissolved into understandable, elegant engineering.

And that is exactly what we did.

- We started with the fundamentals, turning token IDs into meaningful vectors with **Token and Positional Embeddings**.
- We dove deep into the heart of the machine, building the intuition for **Self-Attention** with simple analogies before translating it into the concrete mathematics of Queries, Keys, and Values.
- We made our model practical, implementing the **Causal Mask** to prevent it from seeing the future, and scaling its power with **Multi-Head Attention**.
- We added the "thinking" layer, the **MLP**, and glued everything together with the crucial concepts of **Residual Connections** and **Layer Normalization** to form a complete, stackable `Block`.
- Finally, we assembled the full architecture, understood how it's trained with a **Language Model Head** and a parallelized loss function, and brought it to life with an **Autoregressive Generation** loop.

The `gpt2_min.py` file is no longer a mystery. Every `@torch.no_grad()`, every `.view()`, every `+` sign now has a purpose and a story. The Transformer has clicked.